

Python Programming Mini Project – Personal Expense Tracker

"""

=====

PERSONAL EXPENSE TRACKER

VaultofCodes Python Internship - Week 3 Mini Project

Author : Kanak Chaudhary

Language : Python 3

Features:

- Add Expense
- View Expenses
- Category Summary
- Monthly Summary
- Search Expense
- Edit Expense
- Delete Expense
- JSON File Handling
- Expense Analytics Chart

=====

"""

import json

import os

```
from datetime import datetime
import matplotlib.pyplot as plt
```

```
# =====
```

```
# Configuration
```

```
# =====
```

```
DATA_FILE = "expenses.json"
```

```
# =====
```

```
# Expense Categories
```

```
# =====
```

```
CATEGORIES = [
```

```
    "Food",
```

```
    "Transport",
```

```
    "Shopping",
```

```
    "Entertainment",
```

```
    "Bills",
```

```
    "Health",
```

```
    "Education",
```

```
    "Other"
```

```
]
```

```
# =====
```

```
# Load Expenses
```

```
# =====
```

```

def load_expenses():
    """
    Load expenses from JSON file.
    Returns a list of expense records.
    """

    if not os.path.exists(DATA_FILE):
        with open(DATA_FILE, "w") as file:
            json.dump([], file, indent=4)

        return []

    try:
        with open(DATA_FILE, "r") as file:
            return json.load(file)

    except json.JSONDecodeError:
        print("\n⚠ Corrupted JSON file detected.")
        print("Creating a new expense database...\n")

        with open(DATA_FILE, "w") as file:
            json.dump([], file, indent=4)

        return []

# =====
# Save Expenses
# =====

```

```

def save_expenses(expenses):
    """
    Save expense list into JSON file.
    """

    with open(DATA_FILE, "w") as file:
        json.dump(expenses, file, indent=4)

# =====
# Generate Expense ID
# =====

def generate_id(expenses):
    """
    Generate unique ID for every expense.
    """

    if not expenses:
        return 1

    return max(expense["id"] for expense in expenses) + 1

# =====
# Display Title
# =====

def display_title():
    print("\n" + "=" * 55)

```

```

print("    PERSONAL EXPENSE TRACKER")

print("=" * 55)

# =====

# Pause Screen

# =====

def pause():
    input("\nPress Enter to continue...")

# =====

# Load Existing Data (single load, no duplicate calls below)

# =====

expenses = load_expenses()

# =====

# Add New Expense

# =====

def add_expense():
    """
    Add a new expense to the expense tracker.
    """

    global expenses

    display_title()

```

```
print("Add New Expense\n")
```

```
# -----
```

```
# Amount
```

```
# -----
```

```
while True:
```

```
    try:
```

```
        amount = float(input("Enter Amount (₹): "))
```

```
        if amount <= 0:
```

```
            print("Amount must be greater than zero.\n")
```

```
            continue
```

```
        break
```

```
    except ValueError:
```

```
        print("Invalid amount. Please enter a number.\n")
```

```
# -----
```

```
# Category
```

```
# -----
```

```
print("\nAvailable Categories:\n")
```

```
for index, category_name in enumerate(CATEGORIES, start=1):
```

```
    print(f"{index}. {category_name}")
```

```

while True:

    try:

        choice = int(input("\nChoose Category (1-8): "))

        if 1 <= choice <= len(CATEGORIES):

            category = CATEGORIES[choice - 1]

            break

        else:

            print("Please choose a valid category.")

    except ValueError:

        print("Please enter a valid number.")

# -----
# Description
# -----

description = input("Enter Description: ").strip()

if description == "":

    description = "No Description"

# -----
# Date
# -----

```

```
print("\nDate Options")
```

```
print("1. Use Today's Date & Time")
```

```
print("2. Enter Custom Date & Time")
```

```
while True:
```

```
    choice = input("Choose (1/2): ")
```

```
    if choice == "1":
```

```
        expense_date = datetime.today().strftime("%Y-%m-%d")
```

```
        expense_time = datetime.now().strftime("%H:%M:%S")
```

```
        break
```

```
    elif choice == "2":
```

```
        expense_date = input("Enter Date (YYYY-MM-DD): ")
```

```
        try:
```

```
            datetime.strptime(expense_date, "%Y-%m-%d")
```

```
            expense_time = datetime.now().strftime("%H:%M:%S")
```

```
            break
```



```
except ValueError:

    print("Invalid date format.\n")

else:

    print("Please choose 1 or 2.\n")

# -----
# Create Expense Record
# -----

expense = {

    "id": generate_id(expenses),

    "amount": amount,

    "category": category,

    "description": description,

    "date": expense_date,

    "time": expense_time

}

expenses.append(expense)

save_expenses(expenses)
```

```
print("\nExpense added successfully!")
```

```
print(f"Expense ID : {expense['id']}")
```

```
print(f"Amount   : ₹{amount:.2f}")
```

```
print(f"Category  : {category}")
```

```
print(f>Date     : {expense_date}")
```

```
print(f"Time     : {expense_time}")
```

```
pause()
```

```
# =====
```

```
# View All Expenses
```

```
# =====
```

```
def view_expenses():
```

```
    """
```

```
    Display all saved expenses.
```

```
    """
```

```
    display_title()
```

```
    if not expenses:
```

```
        print("No expenses found.")
```

```
        pause()
```

```
        return
```

```
    print(
```

```
f"{'ID':<5}"
f"{'Amount':<12}"
f"{'Category':<18}"
f"{'Description':<30}"
f"{'Date':<15}"
f"{'Time'}"
)
```

```
print("-" * 80)
```

```
for expense in expenses:
```

```
    print(
        f"{'expense['id']':<5}"
        f"{'₹expense['amount']':<11.2f}"
        f"{'expense['category']':<18}"
        f"{'expense['description']':<30}"
        f"{'expense['date']':<15}"
        f"{'expense.get('time', '--:--:--')}"
    )
```

```
print("-" * 80)
```

```
print(f"Total Records : {len(expenses)}")
```

```
pause()
```

```
# =====
```

```
# Category Summary
```

```
# =====
```

```
def category_summary():
```

```
    display_title()
```

```
    if not expenses:
```

```
        print("No expenses found.")
```

```
        pause()
```

```
        return
```

```
    summary = {}
```

```
    for expense in expenses:
```

```
        category = expense["category"]
```

```
        summary[category] = summary.get(category, 0) + expense["amount"]
```

```
    print("Category-wise Expense Summary\n")
```

```
    print(f"{'Category':<20}{'Total Amount'}")
```

```
    print("-" * 35)
```

```
    for category, total in summary.items():
```

```
        print(f"{category:<20}₹{total:.2f}")
```

```
pause()
```

```
# =====
```

```
# Overall Summary
```

```
# =====
```

```
def overall_summary():
```

```
    display_title()
```

```
    if not expenses:
```

```
        print("No expenses found.")
```

```
        pause()
```

```
        return
```

```
    total = sum(expense["amount"] for expense in expenses)
```

```
    print("Overall Expense Summary\n")
```

```
    print(f"Total Expenses : {len(expenses)}")
```

```
    print(f"Grand Total   : ₹{total:.2f}")
```

```
    pause()
```

```
# =====
```

```
# Monthly Summary
```

```
# =====
```

```
def monthly_summary():
```

```
    display_title()
```

```
    if not expenses:
```

```
        print("No expenses found.")
```

```
        pause()
```

```
        return
```

```
    summary = {}
```

```
    for expense in expenses:
```

```
        month = expense["date"][:7]
```

```
        summary[month] = summary.get(month, 0) + expense["amount"]
```

```
    print("Monthly Expense Summary\n")
```

```
    print(f"{'Month':<15}{'Amount'}")
```

```
    print("-" * 30)
```

```
for month, total in summary.items():
```

```
    print(f"{month:<15}₹{total:.2f}")
```

```
    pause()
```

```
# =====
```

```
# Search Expenses by Category
```

```
# =====
```

```
def search_by_category():
```

```
    """
```

```
    Search and display expenses belonging to a specific category.
```

```
    """
```

```
    display_title()
```

```
    if not expenses:
```

```
        print("No expenses found.")
```

```
        pause()
```

```
        return
```

```
    print("Available Categories:\n")
```

```
    for index, category_name in enumerate(CATEGORIES, start=1):
```

```
        print(f"{index}. {category_name}")
```

```
while True:

    try:

        choice = int(input("\nChoose Category (1-8): "))

        if 1 <= choice <= len(CATEGORIES):
            selected_category = CATEGORIES[choice - 1]
            break

        else:
            print("Please choose a valid category.")

    except ValueError:
        print("Please enter a valid number.")

# Find matching expenses

matching_expenses = []

for expense in expenses:

    if expense["category"] == selected_category:
        matching_expenses.append(expense)

# Display results

print(f"\nExpenses in Category: {selected_category}\n")
```



```
if not matching_expenses:
```

```
    print("No expenses found in this category.")
```

```
    pause()
```

```
    return
```

```
print(
```

```
    f"{'ID':<5}"
```

```
    f"{'Amount':<12}"
```

```
    f"{'Description':<25}"
```

```
    f"{'Date':<16}"
```

```
    f"{'Time'}"
```

```
)
```

```
print("-" * 75)
```

```
category_total = 0
```

```
for expense in matching_expenses:
```

```
    print(
```

```
        f"{expense['id']:<5}"
```

```
        f"₹{expense['amount']:<11.2f}"
```

```
        f"{expense['description']:<25}"
```

```
        f"{expense['date']:<16}"
```

```
f"{expense.get('time', '--:--:--')}"  
)
```

```
category_total += expense["amount"]
```

```
print("-" * 75)
```

```
print(f"Total Records : {len(matching_expenses)}")
```

```
print(f"Total Spent : ₹{category_total:.2f}")
```

```
pause()
```

```
# =====
```

```
# Edit Expense
```

```
# =====
```

```
def edit_expense():
```

```
    """
```

```
    Edit an existing expense using its ID.
```

```
    """
```

```
display_title()
```

```
if not expenses:
```

```
    print("No expenses found.")
```

```
    pause()
```

```
    return
```

```
# Display existing expenses first
```

```
print("Available Expenses:\n")
```

```
for expense in expenses:
```

```
    print(
        f"ID: {expense['id']} | "
        f"₹{expense['amount']:.2f} | "
        f"{expense['category']} | "
        f"{expense['description']} | "
        f"{expense['date']}"
    )
```

```
print()
```

```
# Get Expense ID
```

```
while True:
```

```
    try:
```

```
        expense_id = int(input("Enter Expense ID to edit: "))
```

```
        break
```

```
    except ValueError:
```

```
    print("Please enter a valid numeric ID.")

# Find expense

selected_expense = None

for expense in expenses:

    if expense["id"] == expense_id:

        selected_expense = expense

        break

if selected_expense is None:

    print("\nExpense not found.")

    pause()

    return

# Edit Amount

print("\nPress Enter to keep the existing value.\n")

new_amount = input(

    f"Enter new amount "
```

```
f"(Current: ₹{selected_expense['amount']:.2f}): "  
.strip()
```

```
if new_amount:
```

```
    try:
```

```
        new_amount = float(new_amount)
```

```
        if new_amount > 0:
```

```
            selected_expense["amount"] = new_amount
```

```
        else:
```

```
            print("Invalid amount. Existing amount kept.")
```

```
    except ValueError:
```

```
        print("Invalid amount. Existing amount kept.")
```

```
# Edit Category
```

```
print("\nAvailable Categories:\n")
```

```
for index, category_name in enumerate(CATEGORIES, start=1):
```

```
    print(f"{index}. {category_name}")
```

```
category_choice = input(
    f"\nChoose new category "
    f"(Current: {selected_expense['category']}) "
    f"or press Enter to keep: "
).strip()

if category_choice:

    try:

        category_choice = int(category_choice)

        if 1 <= category_choice <= len(CATEGORIES):

            selected_expense["category"] = (
                CATEGORIES[category_choice - 1]
            )

        else:

            print("Invalid category. Existing category kept.")

    except ValueError:

        print("Invalid category. Existing category kept.")

# Edit Description
```

```
new_description = input(
    f"Enter new description "
    f"(Current: {selected_expense['description']}): "
).strip()

if new_description:

    selected_expense["description"] = new_description

# Edit Date

new_date = input(
    f"Enter new date (YYYY-MM-DD) "
    f"(Current: {selected_expense['date']}): "
).strip()

if new_date:

    try:

        datetime.strptime(new_date, "%Y-%m-%d")

        selected_expense["date"] = new_date

    except ValueError:

        print("Invalid date. Existing date kept.")
```

```
# Save Updated Data
```

```
save_expenses(expenses)
```

```
print("\nExpense updated successfully!")
```

```
pause()
```

```
# =====
```

```
# Delete Expense
```

```
# =====
```

```
def delete_expense():
```

```
    """
```

```
    Delete an existing expense using its ID.
```

```
    """
```

```
display_title()
```

```
if not expenses:
```

```
    print("No expenses found.")
```

```
    pause()
```

```
    return
```



```
# Display expenses
```

```
print("Available Expenses:\n")
```

```
for expense in expenses:
```

```
    print(
        f"ID: {expense['id']} | "
        f"₹{expense['amount']:.2f} | "
        f"{expense['category']} | "
        f"{expense['description']} | "
        f"{expense['date']}"
    )
```

```
print()
```

```
# Get Expense ID
```

```
while True:
```

```
    try:
```

```
        expense_id = int(
            input("Enter Expense ID to delete: ")
        )
```

```
        break
```

```
except ValueError:
```

```
    print("Please enter a valid numeric ID.")
```

```
# Find expense
```

```
selected_expense = None
```

```
for expense in expenses:
```

```
    if expense["id"] == expense_id:
```

```
        selected_expense = expense
```

```
    break
```

```
if selected_expense is None:
```

```
    print("\nExpense not found.")
```

```
    pause()
```

```
    return
```

```
# Show selected expense
```

```
print("\nSelected Expense:")
```

```
print(
    f"ID: {selected_expense['id']}\n"
    f"Amount: ₹{selected_expense['amount']:.2f}\n"
    f"Category: {selected_expense['category']}\n"
    f"Description: {selected_expense['description']}\n"
    f>Date: {selected_expense['date']}"
)

# Confirmation

confirmation = input(
    "\nAre you sure you want to delete this expense? (y/n): "
).strip().lower()

if confirmation == "y":

    expenses.remove(selected_expense)

    save_expenses(expenses)

    print("\nExpense deleted successfully!")

else:

    print("\nDelete operation cancelled.")

pause()
```

```
# =====  
  
# Expense Analytics Chart  
  
# =====  
  
def show_expense_chart():  
    """  
  
    Display a bar chart showing total expenses by category.  
  
    """  
  
    display_title()  
  
    if not expenses:  
        print("No expenses available for chart.")  
        pause()  
        return  
  
    # Calculate total spending for each category  
  
    category_totals = {}  
  
    for expense in expenses:  
  
        category = expense["category"]  
  
        category_totals[category] = (  
            category_totals.get(category, 0)  
            + expense["amount"]  
        )
```

```
# Prepare data for chart
```

```
categories = list(category_totals.keys())
```

```
amounts = list(category_totals.values())
```

```
# Create chart
```

```
plt.figure(figsize=(10, 6))
```

```
plt.bar(categories, amounts)
```

```
plt.title("Expense Summary by Category")
```

```
plt.xlabel("Category")
```

```
plt.ylabel("Total Spending (₹)")
```

```
plt.xticks(rotation=30)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
pause()
```

```
# =====
```

```
# Main Menu
```

```
# =====
```

```
def main():
```

```
    """
```

```
    Run the main menu of the Personal Expense Tracker.
```

```
    """
```

```
    while True:
```

```
        display_title()
```

```
        print("1. Add Expense")
```

```
        print("2. View All Expenses")
```

```
        print("3. View Category Summary")
```

```
        print("4. View Overall Summary")
```

```
        print("5. View Monthly Summary")
```

```
        print("6. Search Expense by Category")
```

```
        print("7. Edit Expense")
```

```
        print("8. Delete Expense")
```

```
        print("9. Show Expense Chart")
```

```
        print("10. Exit")
```

```
        print("=" * 55)
```

```
        choice = input("Enter your choice (1-10): ").strip()
```

```
        if choice == "1":
```

```
add_expense()
```

```
elif choice == "2":
```

```
view_expenses()
```

```
elif choice == "3":
```

```
category_summary()
```

```
elif choice == "4":
```

```
overall_summary()
```

```
elif choice == "5":
```

```
monthly_summary()
```

```
elif choice == "6":
```

```
search_by_category()
```

```
elif choice == "7":
```

```
edit_expense()
```

```
elif choice == "8":
```

```
delete_expense()
```

```
elif choice == "9":
```

```
show_expense_chart()
```

```
elif choice == "10":
```

```
print("\nThank you for using Personal Expense Tracker!")
```

```
print("Your expense data has been saved successfully.")
```

```
break
```

```
else:
```

```
print("\nInvalid choice.")
```

```
print("Please enter a number between 1 and 10.")
```

```
pause()
```

```
# =====
```

```
# Program Entry Point
```

```
# =====
```

```
if __name__ == "__main__":
```


main()